

A Aide-mémoire PL/pgSQL

1. Structure d'une fonction

```
CREATE OR REPLACE FUNCTION nom_fonction(p_param1 INTEGER, p_param2 TEXT)
RETURNS type_de_retour AS $$
DECLARE
    -- déclarations (facultatif)
BEGIN
    -- instructions
    RETURN quelque_chose;
END;
$$ LANGUAGE plpgsql;
```

Les \$\$ délimitent le corps de la fonction (une chaîne de caractères pour PostgreSQL) : à l'intérieur, les apostrophes s'écrivent normalement. Toute instruction se termine par un ;, y compris END IF;, END LOOP; et le END; final. Suppression : DROP FUNCTION nom_fonction(INTEGER, TEXT);

2. Déclarations

```
DECLARE
    v_nb          INTEGER := 0;           -- avec valeur initiale
    v_total        NUMERIC;              -- vaut NULL au départ (piège P2)
    v_titre        film.title%TYPE;      -- même type que la colonne
    v_ligne        film%ROWTYPE;         -- une ligne entière de film
    v_r            RECORD;               -- ligne de structure quelconque
    c_taux         CONSTANT NUMERIC := 0.2; -- constante
```

3. Affectations

```
v_nb := 5;                                -- affectation directe
v_nb := v_nb + 1;
SELECT length INTO v_duree FROM film WHERE film_id = 3;    -- depuis la base
SELECT title, length INTO v_titre, v_duree FROM film WHERE ...; -- plusieurs colonnes
SELECT length INTO STRICT v_duree FROM film WHERE film_id = 3; -- exige 1 ligne
                                exactement
```

Sans STRICT : 0 ligne → variables à NULL, aucune erreur ; plusieurs lignes → la première rencontrée, aucune erreur (P1).

Avec STRICT : 0 ligne → NO_DATA_FOUND ; > 1 ligne → TOO_MANY_ROWS.

FOUND (booléen) indique si la dernière requête a trouvé au moins une ligne – attention, une agrégation en renvoie toujours une (P3). PERFORM requete; exécute une requête dont on ignore le résultat.

4. Conditions

```
IF condition THEN ...
ELSIF autre_condition THEN ...
ELSE ...
END IF;

-- expression CASE (renvoie une valeur)
v_libelle := CASE v_rating WHEN 'G' THEN 'tous publics'
                      WHEN 'R' THEN 'adultes'
                      ELSE 'autre' END;
```

Rappel : IF v IS NULL, jamais IF v = NULL (P4).

5. Boucles

```
LOOP ... EXIT WHEN condition; ... END LOOP;    -- boucle générale
WHILE condition LOOP ... END LOOP;            -- boucle tant que
FOR i IN 1..10 LOOP ... END LOOP;              -- compteur (i est implicite ici)
```

```
FOR i IN REVERSE 10..1 LOOP ... END LOOP;
FOR v_r IN SELECT ... LOOP ... END LOOP;          -- curseur implicite (v_r à déclarer)
```

EXIT sort de la boucle, CONTINUE passe à l'itération suivante.

6. Formes de retour

Déclaration	Instructions	Appel
RETURNS INTEGER	RETURN v;	SELECT f(3);
RETURNS SETOF TEXT	RETURN NEXT v; (n fois) ou RETURN QUERY SELECT ...	SELECT * FROM f(3);
RETURNS TABLE(a INT, b TEXT)	a := ...; b := ...; RETURN NEXT; ou RETURN QUERY SELECT ...	SELECT * FROM f(3);

Les types renvoyés par RETURN QUERY doivent correspondre *exactement* aux types déclarés (d'où les transtypes du genre f.title::TEXT). Une fonction non SETOF qui atteint END sans RETURN lève une erreur à l'exécution (P6).

7. Exceptions

```
RAISE NOTICE 'Valeur : %', v_x;          -- message d'information (n'interrompt rien)
RAISE EXCEPTION 'Client % inexistant', p_id; -- interrompt et annule la transaction

BEGIN
    ...
EXCEPTION
    WHEN division_by_zero THEN ...
    WHEN no_data_found THEN ...
    WHEN too_many_rows THEN ...
    WHEN OTHERS THEN RAISE NOTICE 'Erreur : %', SQLERRM;
END;
```

Un bloc BEGIN ... EXCEPTION ... END peut être imbriqué : placé dans une boucle, il permet de poursuivre le parcours malgré l'erreur d'une itération.

8. Curseurs explicites

```
DECLARE
    c_x CURSOR FOR SELECT ... ;          -- curseur sans paramètre
    c_y CURSOR (p_seuil INTEGER) FOR SELECT ... WHERE length > p_seuil;
    v_r RECORD;
BEGIN
    OPEN c_x;                            -- OPEN c_y(120); pour un curseur paramétré
    LOOP
        FETCH c_x INTO v_r;
        EXIT WHEN NOT FOUND;            -- toujours juste après le FETCH (piège P5)
        ...
    END LOOP;
    CLOSE c_x;
```